Working with Predefined functional interfaces :-

FunctionalInterface functional interface

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a function that takes an argument of type T and returns a result of type R. This is typically used in streams when you need to perform operations that result in primitive values.

@FunctionalInterface
public interface ToIntFunction<T>
{
int applyAsInt(T value);
}

In the same way we have ToLongFunction<T> and ToDoubleFunction<T>.
Predefined functional interfaces :

1) Predicate<T> boolean test(T t)
2) Consumer<T> void accept(T t)
3) Function<T,R> R apply(T t)
4) Supplier<T> T get()
5) BiPredicate<T,U> boolean test(T t, U u)
6) BiConsumer<T,U> void accept(T t, U u)
7) BiFunction<T,U,R> R apply(T t, U u)
8) UnaryOperator<T> T apply(T t)
9) BinaryOperator<T> T apply(T t, T u)
10) ToIntFunction<T> int applyAsInt(T t)
11) ToLongFunction<T> long applyAsLong(T t)
12) ToDoubleFunction<T> double applyAsDouble(T t)

IntStream mapToInt(ToIntFunction<T> mapper)

It is a predefined method of Stream<T> interface.

It is an intermediate operation.

It converts a Stream of objects (Stream<T>) into a primitive IntStream.

IntStream is a specialized stream for working with int values available in java.util.stream sub package.

```
package com.rest.mea.ty_int_long_double;

import java.util.*;
import java.util.stream.*;

public class MapToIntDemo1
{
    public static void main(String[] args)
    {
        List<String> names = Arrays.asList("Ravi", "Amit", "Neha", "Sunil");

        IntStream mapToInt = names.stream().mapToInt(String::length);
        mapToInt.forEach(System.out::print);
    }
}
```

DoubleStream mapToDouble(ToDoubleFunction<T> mapper)

DoubleStream is a specialized stream for working with double values available in java.util.stream sub package.

```
package com.rest.mea.ty_int_long_double;

import java.util.Arrays;
import java.util.List;

public class MapToDoubleDemo1
{
    public static void main(String[] args)
    {
        List<Integer> numbers = Arrays.asList(4, 9, 16, 25);

        numbers.stream()
                .mapToDouble(i -> Math.sqrt(i))
                .forEach(System.out::print);
    }
}
```

LongStream mapToLong(ToLongFunction<T> mapper)

LongStream is a specialized stream for working with long values available in java.util.stream sub package.

```
package com.rest.mea.ty_int_long_double;

import java.util.*;
import java.util.stream.*;

public class MapToLongDemo1
{
    public static void main(String[] args)
    {
        List<String> words = Arrays.asList("Java", "James", "Gosling");

        // Convert strings to long values (length of each word)
        LongStream longs = words.stream()
                                .mapToLong(String::length);

        longs.forEach(System.out::print);
    }
}
```

IntStream interface has provided a predefined abstract methods :

static IntStream range(int startInclusive, int endExclusive) : returns a range of values where startInclusive will be included but endExclusive will be excluded

int sum(): Returns the sum of elements.

OptionalDouble average(): Returns an OptionalDouble.

OptionalInt min(): Returns the min value.

OptionalInt max(): Returns the max value.

long count(): Returns count of elements.

```
package com.rest.mea.ty_int_long_double;

import java.util.Arrays;
import java.util.OptionalDouble;
import java.util.OptionalInt;
import java.util.IntStream.IntStream;

public class IntStreamMethods
{
    public static void main(String[] args)
    {
        IntStream range1, range2, range3, range4, range5;

        int sum1[] = {10,20,30,40,50};

        int sum = Arrays.stream(sum1).sum();
        System.out.println("Sum of all the array elements :"+sum);

        OptionalInt min = Arrays.stream(sum1).min();
        System.out.print("Minimum Element is :");
        min.ifPresent(System.out::print);

        OptionalInt max = Arrays.stream(sum1).max();
        System.out.print("Maximum Element is :");
        max.ifPresent(System.out::print);

        OptionalDouble average = Arrays.stream(sum1).average();
        System.out.print("Average Value is :");
        average.ifPresent(System.out::print);

        long count = Arrays.stream(sum1).count();
        System.out.print("Total Number of Elements :"+count);
    }
}
```

```
package com.rest.mea.ty_int_long_double;

import java.util.stream.Stream;

//Find out the total experience of the Employees
record EmpSalary(String name, int[]exper experience);

{

}

public class IntStreamMethods1
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee("Scott", 12);
        Employee e2 = new Employee("Smith", 15);
        Employee e3 = new Employee("Blake", 12);
        Employee e4 = new Employee("Jones", 20);
        Employee e5 = new Employee("King", 25);
        Employee e6 = new Employee("Ford", 7);

        Set tmpExpr = Stream.of(e1,e2,e3,e4,e5,e6)
                            .map(EmpSalary::experience).set();

        System.out.println("Total Experience of all the employees :"+totalExp);
    }
}
```

```
package com.rest.mea.ty_int_long_double;

import java.util.List;
import java.util.stream.Stream;

//Find the total product price

record ProductIntPrice(int id, String name, Double price)
{
}

public class IntStreamMethods1
{
    public static void main(String[] args)
    {
        Product p1 = new Product(101, "Camera", 12500);
        Product p2 = new Product(102, "Laptop", 34500);
        Product p3 = new Product(103, "Mobile", 15000);
        Product p4 = new Product(104, "Speaker", 8700);
        Product p5 = new Product(105, "Keyboard", 3700);

        List<Product> productList = List.of(p1,p2,p3,p4,p5);

        double totalProducts = productList
                .stream()
                .mapToDouble(Product::price).sum();

        System.out.println("Total price of all the products :"+totalProducts);
    }
}
```

IntStream flatMap(ToIntFunction<T super T, ? extends IntStream> mapper)

It is a predefined method of Stream interface which comes under flatMaping.

It allows us to transform each element of the stream into an IntStream (a stream of primitive int values) and then flatten these streams into a single IntStream.

Note : IntStream is a specialized stream for working with int values available in java.util.stream sub package.

```
package com.rest.basic.flat_map;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class FlatMapToIntStream1
{
    public static void main(String[] args)
    {

        int[] ia = new int[] {1,2,3};
        int[] ib = new int[] {4,5,6};
        int[] ic = new int[] {7,8,9};

        List<int[]> mainArray = Arrays.asList(ia,ib,ic);

        int[] ar = mainArray.stream()
                .flatMapToInt(array -> IntStream.of(array).toArray());
        System.out.println(Arrays.toString(ar));
    }
}
```

LongStream flatMap(ToLongFunction<T super T, ? extends LongStream> mapper)

It is a predefined method of Stream interface which comes under flatMaping.

It allows us to transform each element of the stream into a LongStream (a stream of primitive long values) and then flatten these streams into a single LongStream.

Note : LongStream is a specialized stream for working with long values available in java.util.stream sub package.

```
package com.rest.basic.flat_map;

import java.util.Arrays;
import java.util.List;
import java.util.stream.LongStream;

public class FlatMapToLongStream1
{
    public static void main(String[] args)
    {

        long[] la = new long[] {10,20,30};
        long[] lb = new long[] {40,50,60};
        long[] lc = new long[] {70,80,90};

        List<long[]> longArray = Arrays.asList(la, lb, lc);

        long[] lar = longArray.stream()
                .flatMapToLong(array -> Arrays.stream(array));
        System.out.println(Arrays.toString(lar));
    }
}
```

DoubleStream flatMapToDouble(ToDoubleFunction<T super T, ? extends DoubleStream> mapper)

It is a predefined method of Stream interface which comes under flatMaping.

It allows us to transform each element of the stream into an DoubleStream (a stream of primitive double values) and then flatten these streams into a single DoubleStream.

Note : DoubleStream is a specialized stream for working with double values available in java.util.stream sub package.

```
package com.rest.basic.flat_map;

import java.util.Arrays;
import java.util.List;
import java.util.stream.DoubleStream;

public class FlatMapToDoubleDemo1
{
    public static void main(String[] args)
    {
        double d1[] = new double[]{1.1,2.2,3.3};
        double d2[] = new double[]{4.4,5.5,6.6};
        double d3[] = new double[]{7.7,8.8,9.9};

        List<double[]> listDoubleArray = Arrays.asList(d1,d2,d3);

            DoubleStream doubleStream = listDoubleArray.stream()
                    .flatMapToDoubleArray(a -> DoubleStream.of(a));

            // Print each double value in the flattened stream
            doubleStream.forEach(System.out::print);
    }
}
```

Difference between map() and flatMap():

map() method transforms each element into another single element.

flatMap() transforms each element into a stream of elements and then flatters those streams into a single stream.

We should use map() when you want a one-to-one transformation, and we should use flatMap() when dealing with nested structures or when you need to produce multiple output elements for each input element.

---

public Stream sorted():

It is a predefined method of Stream interface.
It returns a sorted stream.
The return type of this method is Stream.
It has an overloaded method which accept Comparator<T> as a parameter
through which we can provide user defined sorting logic

```
package com.rest.sorted_demo;

import java.util.Arrays;
import java.util.List;

public class SortedDemo1
{
    public static void main(String[] args)
    {
        List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9);

        nums.stream()
                .sorted()
                .forEach(System.out::print);
    }
}
```

```
package com.rest.sorted_demo;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class SortedDemo2
{
    public static void main(String[] args)
    {
        List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9);

        nums.stream()
                .sorted(Comparator.reverseOrder())
                .forEach(System.out::print);
    }
}
```

Comparator interface has provided the following static methods :-

1) public static Comparator comparing(ToDoubleFunction keyExtractor)
2) public static Comparator comparingInt(ToFunction keyExtractor)
3) public static Comparator comparingLong(ToLongFunction keyExtractor)
4) public static Comparator comparingDouble(ToDoubleFunction keyExtractor)

```
package com.rest.sorted_demo;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.stream.Stream;

record StudentI(Integer id, String name)

public class SortedDemo3
{
    public static void main(String[] args)
    {
        List<StudentI> students = Arrays.asList(
                new StudentI(5, "Scott"),
                new StudentI(2, "Smith"),
                new StudentI(1, "Blake")
                );

        // Sort by name
        Stream<StudentI> sorted = students.stream()
                                          .sorted(Comparator.comparing(StudentI::name));

        sorted.forEach(System.out::print);
    }
}
```

Stream<T> boxed() :-

It is a predefined method of IntStream through which we can convert IntStream to Stream

```
package com.rest.sorted_demo;

import java.util.Arrays;
import java.util.CollectionUtils;
import java.util.Comparator;

//Sort the Array Data in descending order
public class SortedDemo4
{
    public static void main(String[] args)
    {
        Integer [] arr = {10,50, 30, 90, 70, 70};
        Arrays.stream(arr)
                .sorted(Comparator.reverseOrder())
                .forEach(System.out::print);

        System.out.println("----------------------------");

        int []nrs = {10,50,30,90,70};

        Arrays.stream(nrs).boxed()
                .sorted(Collections.reverseOrder())
                .forEach(System.out::print);
    }
}
```